Various Notes:

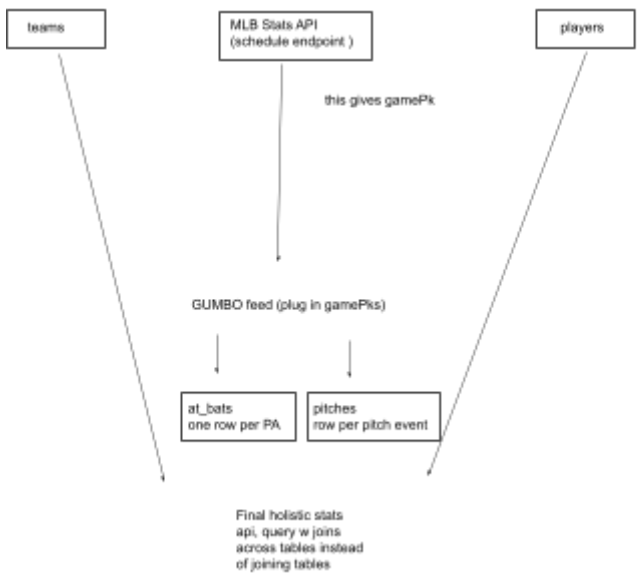
API Sources:

- Team → team_id, name, league, division
- Person → player_id, name, birth_date
- Sports → game_pk, date, home, away
- Schedule →
- GUMBO → This is pitch level
    - at bats and pitches (speed, zone, exit_velo)

the path to a pitch is liveData → plays → allPlays → [each play] → playEvents → [each event]. Not every playEvent is a pitch — filter on details.type.code or check that pitchData exists on the event before trying to read speed or zone.

test api calls on GUMBO →

allPlays is an array of each atBat of the game



Type	Req, return description	How
Game	**Description:** This endpoint returns the Gumbo Live Feed for a specific gamePk. **Return Includes:** Team information, live play by play data, and player information. **Required Parameters:** gamePk is required to run this call. **Hydrations:** This endpoint can accept the hydrations query parameter.	

Schedule	**Description:** This endpoint returns all schedules based on a particular scheduleType. **Return Includes:** Game and event data. **Required Parameters:** sportId or gamePk(s) are required to run this call **Hydrations:** This endpoint can accept the hydrations query parameter.	
team	**Description:** This endpoint returns team information based on year,leagueId,sportId and gameType. **Return Includes:** League,division,sport and venue information for each team. **Required Parameters:** No parameters are required to run this call. However, sportIds and leagueIds must be called seperately **Hydrations:** This endpoint can accept the hydrations query parameter.	"sportId": 1, "season": season, "hydrate": "league,division"
Person	**Description:** This endpoint returns statistical data and biographical information for players,umpires, and coaches based on playerId. **Return Includes:** personId, DOB, statistics for players based on statType. **Required Parameters:** personIds is required to run this call. **Hydrations:** This endpoint can accept the hydrations query parameter.	
Gumbo	On GUMBO's structure — the path to a pitch is liveData → plays → allPlays → [each play] → playEvents → [each event]. Not every playEvent is a pitch — filter on details.type.code or check that pitchData exists on the event before trying to read speed or zone. Games with rain delays, mid-inning pitching changes, or incomplete data will have weird gaps, so always use .get() in Python rather than direct key access.	

Use sqlite for project

- Pitch level data may be a lot but able to be handled in two seasons
- Local

teams
↑
games
↑
at_bats
↑
pitches
↓
players

Exit velocity only gets recorded on the one pitch per at-bat where the batter actually puts the ball in play — a single, double, home run, groundout, flyout, etc. So in a typical at-bat that goes ball, strike, foul, foul, groundout — only that last pitch has hitData.

SQLite stores None and NULL

id fields are the only things that should be NOT NULL because I don’t want to drop the entire insert if some things that are sometimes occasionally missing via my test pulls

I think it needs to be pulled in order so that there is access to game pks and player info

start with teams, largest layer

get gamepks

get players

read game pks and call gumbo per game pk

apeksha@Apekshas-MacBook-Pro-1544 orioles_eda % sqlite3 src/database/mlb.db "SELECT COUNT(*) FROM teams;"

2025 season is overwriting the 2024 rows because team_id alone is the primary key, same team IDs exist in both seasons so insert OR replace replaces them.
Your schema needs a composite primary key of team_id + season together, not just team_id alone.

Changed from:
team_id INTEGER PRIMARY KEY NOT NULL
to PRIMARY KEY (team_id, season)

> Once teams got a composite primary key, the games table broke because its foreign key was pointing at teams(team_id) alone, which no longer exists as a standalone primary key. Removed those foreign key constraints from games entirely since they add complexity without much benefit for bulk ingestion.

Always drop tables in child-to-parent order before recreating schema — pitches → at_bats → games → players → teams. SQLite won't let you drop a parent table while a child table still has a foreign key pointing at it.

officialDate is different than gameDate (timezone attached)

make sure we pull on finalized games

abstractGameState

- Preview
- Live
- Final

detailedState

- Warmup
- On Progress
- Delayed

gamePk gumbo

game → liveData → plays → allPlays → play → playEvents → event

- an atBat is a batter's entire turn at the plate
- a pitch is one single throw within the at-bat

```
game

└── plays (list of at-bats)

    └── playEvents (list of pitch events)

        └── pitchData (the actual pitch metrics)
```

game is one full mlb game

play is one plate appearance

result is metadata about the at-bat

event is or is not a pitch

pitchData is statcast level physics data

```
Game (container)

 ├── At-bats (decisions / outcomes)

 │    ├── Events (actions inside at-bat)
```

```
│   │   ├── Pitches (physics data)
│   │   ├── Fouls
│   │   └── Ball/Strike calls
```

Only storing complete at bats

Process was crashing with incomplete at bats

top of inning = away team is batting, home team is pitching

bottom of inning = home team is batting, away team is pitching

TESTING API CALLS

```
r = requests.get(f'https://statsapi.mlb.com/api/v1.1/game/{game_pk}/feed/live')

if r.status_code != 200:

    return
```

Hits GUMBO for one specific game. If the API returns anything other than 200 it logs the failure and moves on to the next game rather than crashing the whole pipeline.

```
plays = data.get("liveData", {}).get("plays", {}).get("allPlays", [])
```

Drills three levels deep into the GUMBO response to get the list of at-bats. The chained .get() with empty dict defaults means if any level is missing it just returns an empty list rather than crashing.

At-bat level

```
for play in plays:

    at_bat_id = f"{game_pk}_{at_bat_index}"

    batter_id = play.get("matchup", {}).get("batter", {}).get("id")

    pitcher_id = play.get("matchup", {}).get("pitcher", {}).get("id")
```

Loops through every at-bat in the game. Constructs a unique at_bat_id by combining game_pk and atBatIndex since the API doesn't give you one. Safely pulls batter and pitcher IDs from the matchup object.

Team ID

```
if play["about"]["halfInning"] == "top":

    batter_team_id  = away_team_id

    pitcher_team_id = home_team_id

else:

    batter_team_id  = home_team_id

    pitcher_team_id = away_team_id
```

GUMBO doesn't tell you which team a player is representing in a given play. You figure it out from baseball rules — away team always bats in the top half of an inning, home team always bats in the bottom half. You already have home_team_id and away_team_id from your games table, passed in as function arguments.

```
conn.execute("""INSERT OR REPLACE INTO at_bats...""")
```

Writes one row to at_bats for this play — who batted, who pitched, which inning, which half, what the result was. Wrapped in its own try/except so a single bad at-bat doesn't kill the whole game.

```
for event in play.get("playEvents", []):

    if not event.get("isPitch", False):

        continue
```

Loops through every event inside this at-bat. Not every event is a pitch — there are pickoff attempts, mound visits, substitutions. The isPitch filter skips all of those and only processes real pitches.

Pulling pitch fields

```
pitch_data = event.get("pitchData", {})

hit_data = event.get("hitData", {})

details = event.get("details", {})
```

Three separate objects inside each pitch event. pitchData has speed and zone. hitData only exists when the ball was put in play — most pitches it's empty. details has pitch type and call.

```
conn.execute("""INSERT OR REPLACE INTO pitches...""")
```

Writes one row to pitches per pitch. Constructs pitch_id from game_pk, at_bat_index, and pitch_number. Uses .get() on every Statcast field so missing data stores as NULL rather than crashing.

```
conn.commit()

finally:

    conn.close()
```

Commits all the inserts for the entire game in one transaction after all plays and pitches are processed. The finally block ensures the connection always closes even if something throws an exception mid-game.

```
for game in games:

    if game_pk in completed:

        continue

    fetch_gumbo(...)

    save_checkpoint(game_pk)

    time.sleep(1)
```

Reads all 4,860 game records from your games table, skips already-completed ones, calls fetch_gumbo for each remaining game, checkpoints on success, and waits one second between calls to respect the API rate limit.

Q1 — pitches over 95mph on July 8, 2025

sql

```sql
SELECT p.full_name, pi.start_speed, t.name

FROM pitches pi

JOIN players p ON pi.pitcher_id = p.player_id

JOIN teams t ON pi.pitcher_team_id = t.team_id AND t.season = 2025

WHERE pi.game_date LIKE '2025-07-08%' AND pi.start_speed > 95

ORDER BY pi.start_speed DESC

LIMIT 10;
```

Start from pitches — that's where speed lives. Join to players to get the pitcher's name from their ID. Join to teams to get the team name, filtering on season = 2025 so you don't accidentally match the same team ID from 2024. Filter to only that date using LIKE because your dates have timestamps attached. Filter to only pitches over 95mph. Sort fastest first.

Q2 — season percentage over 95mph for those pitchers

sql

```sql
SELECT p.full_name,

    COUNT(*) as total_pitches,

    SUM(CASE WHEN pi.start_speed > 95 THEN 1 ELSE 0 END) as over_95,

    ROUND(100.0 * SUM(CASE WHEN pi.start_speed > 95 THEN 1 ELSE 0 END) / COUNT(*), 1)
AS pct_over_95

FROM pitches pi

JOIN players p ON pi.pitcher_id = p.player_id

WHERE pi.season = 2025

AND pi.pitcher_id IN (
```

```
    SELECT DISTINCT pitcher_id FROM pitches

    WHERE game_date LIKE '2025-07-08%'

    AND start_speed > 95

)

GROUP BY pi.pitcher_id

ORDER BY pct_over_95 DESC;
```

The inner query (IN subquery) finds the distinct pitcher IDs who threw over 95mph on July 8 — that's your target group. The outer query then looks at all their pitches for the entire 2025 season. COUNT(*) counts every pitch they threw. SUM(CASE WHEN start_speed > 95 THEN 1 ELSE 0 END) counts only the ones over 95 — the CASE WHEN is essentially an if/else that returns 1 for true and 0 for false, so summing it counts the trues. Dividing those two gives the percentage. GROUP BY pitcher_id means you get one row per pitcher.

Q3 — top 10 hitters by exit velocity on high fastballs

sql

```sql
SELECT p.full_name,

      ROUND(AVG(pi.hit_speed), 1) AS avg_exit_velo,

      COUNT(*) as balls_in_play

FROM pitches pi

JOIN players p ON pi.batter_id = p.player_id

WHERE pi.pitch_type IN ('FF','SI','FC')

AND pi.zone IN (1,2,3)

AND pi.hit_speed IS NOT NULL

AND pi.season = 2025

GROUP BY pi.batter_id
```

```
HAVING COUNT(*) >= 10

ORDER BY avg_exit_velo DESC

LIMIT 10;
```

Filter to fastball pitch types — FF is four-seam, SI is sinker, FC is cutter. Filter to zones 1, 2, 3 which are the top row of the strike zone from the PlateZones diagram. Filter to only pitches where hit_speed IS NOT NULL — meaning the ball was actually put in play. AVG(hit_speed) gives average exit velocity per batter. HAVING COUNT(*) >= 10 removes batters with too few samples to be meaningful — you need at least 10 balls in play for the average to mean anything. Sort by highest average exit velocity.

Q4 — percentile for a specific hitter among under-26 players

sql

```SQL
```

The WITH block is called a CTE — Common Table Expression. Think of it as a temporary table you define at the top and then query against. It builds a list of every qualifying batter under 26 and their average exit velocity. The age calculation julianday(game_date) - julianday(birth_date)) / 365.25 converts both dates to a number of days, subtracts them, then divides by 365.25 to get years — that's how you compute age in SQLite since it has no built-in age function.

The outer query then counts how many batters in that list have a lower average exit velocity than Gunnar Henderson (batter_id 683002). Dividing that count by the total count and multiplying by 100 gives his percentile. So 91.7 means he hits harder than 91.7% of qualifying under-26 hitters.

```
sqlite> SELECT p.full_name, pi.start_speed, t.name
   ...> FROM pitches pi
   ...> JOIN players p ON pi.pitcher_id = p.player_id
   ...> JOIN teams t ON pi.pitcher_team_id = t.team_id AND t.season = 2025
   ...> WHERE pi.game_date LIKE '2025-07-08%' AND pi.start_speed > 95
   ...> ORDER BY pi.start_speed DESC
   ...> LIMIT 10;
Jacob Misiorowski|101.6|Milwaukee Brewers
Jacob Misiorowski|101.5|Milwaukee Brewers
Jacob Misiorowski|101.4|Milwaukee Brewers
Trevor Megill|101.3|Milwaukee Brewers
Jacob Misiorowski|101.1|Milwaukee Brewers
Jacob Misiorowski|100.8|Milwaukee Brewers
Jacob Misiorowski|100.8|Milwaukee Brewers
Jacob Misiorowski|100.7|Milwaukee Brewers
Jacob Misiorowski|100.7|Milwaukee Brewers
Jacob Misiorowski|100.6|Milwaukee Brewers
sqlite>
```

```
SELECT p.full_name,
```

```
    COUNT(*) as total_pitches,

    SUM(CASE WHEN pi.start_speed > 95 THEN 1 ELSE 0 END) as over_95,

    ROUND(100.0 * SUM(CASE WHEN pi.start_speed > 95 THEN 1 ELSE 0 END) /
COUNT(*), 1) AS pct_over_95

FROM pitches pi

JOIN players p ON pi.pitcher_id = p.player_id

WHERE pi.season = 2025

AND pi.pitcher_id IN (

  SELECT DISTINCT pitcher_id FROM pitches

  WHERE game_date LIKE '2025-07-08%'

  AND start_speed > 95

)

GROUP BY pi.pitcher_id

ORDER BY pct_over_95 DESC;
```

Emmanuel Clase|737|513|69.6 Will Vest|1126|753|66.9 Juan Morillo|617|405|65.6 Jacob Misiorowski|1161|722|62.2 Eduarniel Núñez|249|155|62.2 Trevor Megill|753|456|60.6 Juan Mejia|1074|649|60.4 Félix Bautista|627|370|59.0 Brock Stewart|610|360|59.0 Angel Zerpa|1086|637|58.7 Seranthony Domínguez|1102|625|56.7 Gregory Soto|1031|571|55.4 Abner Uribe|1164|628|54.0 Lucas Erceg|946|511|54.0 Ryan Walker|1018|528|51.9 Eury Pérez|1673|867|51.8 Orion Kerkering|1073|544|50.7 Edwin Díaz|1066|532|49.9 Camilo Doval|1104|548|49.6 Kaleb Ort|788|379|48.1 Ryan Helsley|979|447|45.7 Jacob deGrom|2614|1194|45.7 Bryan Baker|1099|496|45.1 Huascar Brazobán|1011|447|44.2 Dan Altavilla|456|200|43.9 Alex Carrillo|84|35|41.7 Jared Koenig|1043|420|40.3 Porter Hodge|594|235|39.6 Carlos Estévez|1043|402|38.5 Reid Detmers|1093|376|34.4 Wandy Peralta|1139|377|33.1 Sam Bachman|335|109|32.5 Ian Hamilton|641|201|31.4 Lyon Richardson|674|211|31.3 Yennier Cano|986|307|31.1 Cristopher Sánchez|2896|855|29.5 Griffin Jax|1093|321|29.4 Brayan Bello|2697|757|28.1 Reed Garrett|979|273|27.9 Ryan Pepiot|2699|719|26.6 Mason Thompson|239|62|25.9 Andrew Kittredge|768|197|25.7 Juan Burgos|220|56|25.5 Kevin

Ginkel|491|122|24.8 Tyler Phillips|1114|274|24.6 Logan Gilbert|2156|515|23.9 Zach Agnos|524|122|23.3 Max Lazar|646|117|18.1 Victor Mederos|314|54|17.2 Casey Legumina|861|142|16.5 Yusei Kikuchi|3094|490|15.8 Chris Martin|638|88|13.8 Dennis Santana|998|131|13.1 Tanner Bibee|2970|341|11.5 Geoff Hartlieb|109|11|10.1 Jake Bird|951|80|8.4 Mitch Keller|2870|230|8.0 Clay Holmes|2764|151|5.5 JoJo Romero|990|50|5.1 Brandon Young|940|43|4.6 Edwin Uceta|1214|53|4.4 Joey Wentz|1781|72|4.0 Zac Gallen|3135|117|3.7 Yu Darvish|1158|38|3.3 Will Warren|2802|83|3.0 Simeon Woods Richardson|1878|53|2.8 Jack Flaherty|2831|60|2.1 Jake Irvin|2945|24|0.8

```
SELECT p.full_name,
    ROUND(AVG(pi.hit_speed), 1) AS avg_exit_velo,
    COUNT(*) as balls_in_play
FROM pitches pi
JOIN players p ON pi.batter_id = p.player_id
WHERE pi.pitch_type IN ('FF','SI','FC')
AND pi.zone IN (1,2,3)
AND pi.hit_speed IS NOT NULL
AND pi.season = 2025
GROUP BY pi.batter_id
HAVING COUNT(*) >= 10
ORDER BY avg_exit_velo DESC
LIMIT 10;
SELECT p.full_name,
```

```
    ROUND(AVG(pi.hit_speed), 1) AS avg_exit_velo,
    COUNT(*) as balls_in_play
FROM pitches pi
JOIN players p ON pi.batter_id = p.player_id
WHERE pi.pitch_type IN ('FF','SI','FC')
AND pi.zone IN (1,2,3)
AND pi.hit_speed IS NOT NULL
AND pi.season = 2025
GROUP BY pi.batter_id
HAVING COUNT(*) >= 10
ORDER BY avg_exit_velo DESC
LIMIT 10;
```

James Wood|102.4|30 Shohei Ohtani|99.4|65 Romy Gonzalez|99.2|22 Juan Soto|98.7|60 Roman Anthony|98.0|26 Alejandro Kirk|97.8|50 Manny Machado|97.5|39 Jeremiah Jackson|97.4|18 Gunnar Henderson|97.2|50 Ryan Mountcastle|97.2|26

```
WITH exit_velos AS (
```

```
  SELECT pi.batter_id,
        AVG(pi.hit_speed) AS avg_exit_velo
  FROM pitches pi
  JOIN players p ON pi.batter_id = p.player_id
  WHERE pi.hit_speed IS NOT NULL
  AND pi.season = 2025
  AND (julianday(pi.game_date) - julianday(p.birth_date)) / 365.25 < 26
  GROUP BY pi.batter_id
  HAVING COUNT(*) >= 10
)
SELECT
  ROUND(100.0 * SUM(CASE WHEN avg_exit_velo < (
     SELECT avg_exit_velo FROM exit_velos WHERE batter_id = 683002
  ) THEN 1 ELSE 0 END) / COUNT(*), 1) AS percentile
FROM exit_velos;
```

91.7

```
orioles_eda % sqlite3 src/database/mlb.db "SELECT season, COUNT(*) FROM pitches GROUP BY season;"

2024|709511

2025|709075

apeksha@Apekshas-MacBook-Pro-1544 orioles_eda %
```